

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTMENT OF COMPUTER SCIENCE

Object-Oriented Programming

Homework 6- Software Requirements Specification (SRS).

Group N°1

Project: Liquor Store System

Team Name: Logical Lords

Members:

Isaac Sebastian Acurio Catota

Armas Guaño Demian Rolando

Enrique Mateo Ayala Lagunas

Instructor: Edison Lascano PhD

NRC: 36862

October 15, 2026

Quito – Ecuador

Table of Contents

1. Product Perspective and Operational Context 4
   1.1 Purpose 4
   1.2 Product Context 4
   1.3 First Version Scope 4
   1.4 Users 4
2. Operating Conditions and Development Constraints 5
   2.1 Development Environment 5
   2.2 Data Persistence 5
   2.3 Sales and Payments 5
   2.4 Prices and Gross Profit 5
   2.5 Verification Approach 6
3. Business Rules 6
   3.1 Monetary Values and Rounding 6
   3.2 Products and Stock 6
   3.3 Low Stock Alerts 7
   3.4 Suppliers and Purchase Orders 7
   3.5 Sales Registration and Confirmation 7
   3.6 Cash Sales and Customer Credit 8
   3.7 Percentage Discounts 8
   3.8 Gross Profit and Gross Profit Margin 8
   3.9 Daily Cash Reconciliation 9
   3.10 Best Selling Products 9
   3.11 Age Verification 10
4. Functional Requirements 10
5. Non-Functional Requirements 12
   5.1 Reference Test Conditions 12
   5.2 Quality Requirements 12
6. External Interface Requirements 13
   6.1 User Interface 14
      6.1.1 Main Menu and Navigation 14
      6.1.2 Input Formats 14
      6.1.3 Console Outputs 15
      6.1.4 Sales Workflow 15
   6.2 Hardware Interfaces 16
   6.3 Software Interfaces 16

6.4 Local Data File Interface 16
6.5 Communication Interfaces 16
7. Data Requirements 17
7.1 Data Types and Conventions 17
7.2 Minimum Data Records 17
7.3 Relationships and Historical Values 18
7.4 Calculated Information 18
7.5 JSON File Organization 19
7.6 Loading, Saving, and Data Validation 19
8. Assumptions, Dependencies and Limitations 20
8.1 Assumptions 20
8.2 Software and Environment Dependencies 20
8.3 First Version Limitations 21
8.3.1 Interaction and Access 21
8.3.2 Inventory and Supplier Operations 21
8.3.3 Sales and Payments 21
8.3.4 Financial Results 21
8.3.5 Data Storage 22
8.4 Requirements Changes 22
9. Verification and Requirements Traceability 22
9.1 Verification Methods 22
9.2 Unit Test Planning and Responsibilities 23
9.3 Functional Requirements Traceability 23
9.4 Shared Requirements Verification 24
9.5 Non-Functional Requirements Verification 25
9.6 Test Records and Acceptance 25
10. References and Appendices 26
10.1 References 26
10.2 Appendix A Glossary 26
10.3 Supporting Appendices 27

Software Requirements Specification — Liquor Store System”**.**

1. Product Perspective and Operational Context

1.1 Purpose

This document specifies the requirements for the first version of Liquor Store System. It describes the system’s expected behavior, operating conditions, business rules, and verification criteria.

The specification is based on the Project Definition and the ten user stories in the Product Backlog. It will guide the implementation, testing, and validation of the application with the client.

1.2 Product Context

Liquor Store System is an academic project developed for a real liquor store managed by its owner with the support of an employee. The business currently combines manual records with information kept on a mobile phone.

The application will centralize information about products, inventory, suppliers, purchase orders, sales, customer credit, payments, and cash movements. Its main purpose is to support accurate records and provide information for daily management decisions.

1.3 First Version Scope

The first version will provide the following capabilities:

- Register products and manage stock entries, exits, and available quantities.
- Identify products whose stock reaches or falls below a defined minimum.
- Register suppliers, record purchase orders, and confirm deliveries.
- Record sales and identify the owner or employee responsible for each transaction.
- Record customer credit purchases, payments, and outstanding balances.
- Calculate expected closing cash and compare it with the amount physically counted.
- Calculate gross profit and gross profit margins.
- Apply percentage discounts entered during a sale.
- Report best-selling products during a selected period.
- Display an age-verification reminder and request the cashier’s confirmation before completing a sale.

The application will initially support one liquor store and one computer.

1.4 Users

The owner manages business information, reviews sales and customer balances, and performs daily cash reconciliation.

The cashier is the person recording a sale. This responsibility may be performed by the owner or the employee. Each completed sale must identify its responsible seller.

2. Operating Conditions and Development Constraints

2.1 Development Environment

The application shall be implemented in Java using NetBeans and object-oriented programming principles.

The first version shall use a console interface with numbered menus, text prompts, and readable results. The user shall be able to enter information, perform operations, consult records, and return to the main menu.

Project documentation and source-code identifiers shall use English. Client-facing menus, prompts, messages, and results shall use Spanish. Source code shall be written without comments.

2.2 Data Persistence

The application shall store its records in local JSON files and load previously saved information when it starts.

Successful changes to products, suppliers, orders, sales, customer accounts, payments, and cash records shall be persisted. Invalid or canceled operations shall leave the existing records unchanged.

The application shall operate as a single active session on one computer.

2.3 Sales and Payments

Sales shall be recorded as either cash or credit transactions.

A cash sale represents money received when the sale is completed. A credit sale creates an outstanding customer balance. Subsequent customer payments reduce that balance and are recorded separately from sales.

Discounts shall be entered as percentages applied to the total sale amount, following the rules authorized by the owner.

2.4 Prices and Gross Profit

The user shall enter each product’s purchase cost per unit and labeled selling price.

Gross profit shall be calculated from the actual amount charged after discounts and the purchase cost of the units sold. The gross profit margin shall express that profit as a percentage of the actual sale amount.

A purchase cost of $8 and a selling price of $10 produce $2 of gross profit and a 20% gross profit margin. The system shall calculate the resulting margin without automatically changing the labeled selling price.

2.5 Verification Approach

Business calculations and validation rules shall be organized into methods that can be called from unit tests without requiring manual console input.

Console interactions and JSON persistence shall also be checked to verify that the application can complete its workflows and recover saved information after restarting.

3. Business Rules

3.1 Monetary Values and Rounding

Monetary amounts shall be expressed in US dollars. Monetary inputs shall contain no more than two decimal places.

Calculated monetary amounts and displayed percentages shall be rounded to two decimal places using the half-up rule. For example, 1.235 shall be rounded to 1.24.

Invalid values shall produce an explanatory message and allow correction before the operation is completed.

3.2 Products and Stock

Each product shall have a unique, nonempty code and a name. Its available quantity shall be a whole number equal to or greater than zero.

Purchase costs per unit shall be equal to or greater than zero. Labeled selling prices shall be greater than zero.

Stock entries and exits shall require positive whole quantities. An exit shall be rejected when its quantity exceeds the available stock.

When the same product appears more than once in a sale, the total requested quantity for that product shall be checked against its available stock.

3.3 Low Stock Alerts

Each product shall have a minimum stock level expressed as a whole number equal to or greater than zero.

A product shall be identified as having low stock when:

Available stock ≤ Minimum stock level

The console shall identify products meeting this condition when inventory is consulted or updated. Alerts shall reflect the current stock quantities.

3.4 Suppliers and Purchase Orders

Each purchase order shall identify an existing supplier and contain at least one registered product with a positive whole quantity.

Orders shall have two statuses in the first version: Pending and Received.

A new order shall begin as Pending. Confirming receipt shall add the ordered quantities to inventory and change its status to Received.

For this version, confirmation shall represent receipt of the complete order. An order already marked as Received shall not increase stock again.

3.5 Sales Registration and Confirmation

A sale shall contain at least one registered product, positive whole quantities, a date, and an identified seller.

The seller shall be identified as the owner or employee responsible for the transaction.

The sale subtotal shall be calculated as:

Subtotal = Sum of quantity × selling price per unit

A completed sale shall store its products, quantities, purchase costs per unit, selling prices per unit, discount, final amount, payment type, date, and seller.

Later changes to product information shall not alter previously recorded sale amounts or gross profit.

The application shall validate the sale before recording it and reducing inventory. Canceling or rejecting a sale shall leave stock, customer balances, and cash records unchanged.

3.6 Cash Sales and Customer Credit

A cash sale shall represent payment of the complete final sale amount when the transaction is completed.

A credit sale shall require an identified customer and increase that customer's outstanding balance by the complete final sale amount.

Subsequent customer payments shall be recorded separately with their amount and date. Each payment shall reduce the customer's outstanding balance.

Payment amounts shall be greater than zero and shall not exceed the outstanding balance. A full payment shall leave a balance of zero.

A customer payment shall not create another sale or reduce inventory again.

3.7 Percentage Discounts

The cashier shall enter a discount percentage according to the owner's authorization. The valid range shall be:

$0 \leq \text{Discount percentage} < 100$

The discount shall apply to the complete sale subtotal.

$\text{Discount amount} = \text{Subtotal} \times \text{Discount percentage} \div 100$

The calculated discount amount shall be rounded to two decimal places. The final amount shall then be calculated as:

$\text{Final sale amount} = \text{Subtotal} - \text{Rounded discount amount}$

A 0% discount shall leave the subtotal unchanged. The final sale amount shall be greater than zero.

The discount percentage and discount amount shall be stored with the completed sale.

3.8 Gross Profit and Gross Profit Margin

Gross profit shall use the actual final sale amount after discounts and the purchase costs stored with the sale.

$\text{Total purchase cost} = \text{Sum of quantity} \times \text{purchase cost per unit}$

$\text{Gross profit} = \text{Final sale amount} - \text{Total purchase cost}$

Gross profit margin = Gross profit ÷ Final sale amount × 100

A margin calculation shall be rejected if the final sale amount is zero or negative.

Buying one unit for $8 and selling it for $10 shall produce $2 of gross profit and a 20% gross profit margin.

If a 10% discount is applied to that sale, the final amount shall be $9, gross profit shall be $1, and the displayed margin shall be 11.11%.

The expected 20% margin shall serve as a reference for evaluating the calculated result. It shall not automatically change the labeled selling price or prevent a sale with a different margin.

Gross profit shall be calculated before deducting the store's other operating expenses.

3.9 Daily Cash Reconciliation

Each daily reconciliation shall use an opening cash balance and the cash operations recorded for the selected date.

Opening cash and physically counted cash shall be equal to or greater than zero. Cash expenses shall be greater than zero.

Expected closing cash shall be calculated as:

Expected cash = Opening cash + Cash sales + Customer debt payments − Cash expenses

Only cash sales, customer payments, and expenses recorded on the selected date shall affect that day's calculation.

Cash receipts from sales and customer payments shall be obtained from their recorded operations and shall not be entered again as additional income.

Unpaid credit sales shall not increase expected cash.

Cash difference shall be calculated as:

Cash difference = Counted cash − Expected cash

A positive difference shall indicate a surplus. A negative difference shall indicate a shortage. A difference of zero shall indicate that counted and expected cash agree.

3.10 Best Selling Products

The reporting period shall have a start date and an end date. The start date shall be earlier than or equal to the end date.

Both boundary dates shall be included.

The report shall total the units sold for each product in completed sales during the selected period, including credit sales.

Customer debt payments shall not add units to the report.

Products shall be listed in descending order of units sold. If there are no completed sales in the period, the console shall display an informative message.

3.11 Age Verification

Before completing each cash or credit sale, the application shall display an age-verification reminder.

The cashier shall confirm that the age check has been performed or cancel the sale.

A sale shall not be completed without this confirmation. Canceling at this step shall leave inventory, customer balances, and cash records unchanged.

4. Functional Requirements

The following requirements define the behavior of the first version of Liquor Store System. Each requirement shall follow the applicable business rules established in Section 3.

Requirement ID	Backlog Reference	Functional Requirement
FR-01	US001	Product and stock management. The system shall allow the user to register, view, and update products using a unique code, name, purchase cost per unit, labeled selling price, and available quantity. It shall allow stock entries and exits and display the resulting quantities. Stock operations shall follow the validations in Section 3.2.
FR-02	US002	Low-stock alerts. The system shall allow the owner to set a minimum stock level for each product. When inventory is consulted or updated, the console shall identify products whose available quantity is equal to or below that level. The displayed alerts shall reflect current quantities.
FR-03	US003	Suppliers and purchase orders. The system shall allow the user to register and consult suppliers with an identifier, name, and contact information. It shall allow the creation and consultation of purchase orders containing a supplier, products, quantities, and status. Confirming receipt of a Pending order shall increase stock and mark the order as Received. Repeated confirmation

		shall not increase stock again.
FR-04	US004	Sales registration and seller tracking. The system shall allow the user to register cash or credit sales, identify the responsible seller, and enter products and quantities. It shall calculate the subtotal and final amount, validate stock availability, and record the transaction when confirmed. Completed sales shall reduce inventory and preserve the costs, prices, quantities, discount, payment type, date, and seller used in the transaction. The owner shall be able to consult sales by responsible seller.
FR-05	US005	Customer credit and payments. The system shall allow the user to register customers with a unique identifier and name and associate credit sales with them. It shall record partial or full payments and update outstanding balances according to Section 3.6. The owner shall be able to consult each customer's credit purchases, payment history, and remaining balance.
FR-06	US006	Daily cash reconciliation. The system shall allow the user to record an opening cash balance, cash expenses, and physically counted cash for a selected date. It shall calculate expected closing cash using that day's cash sales, customer debt payments, and expenses. The report shall display the date, expected cash, counted cash, and resulting surplus or shortage according to Section 3.9.
FR-07	US007	Gross profit and margin. The system shall calculate and display gross profit and gross profit margin for completed sales using the actual amounts charged after discounts and the purchase costs stored with each sale. Calculations shall follow Section 3.8. The expected 20% margin shall serve as a reference without automatically modifying labeled selling prices. Later product updates shall not alter historical results.
FR-08	US008	Percentage discounts. The system shall allow the cashier to enter a discount percentage according to the owner's authorization. It shall validate the percentage, calculate the discount amount, and calculate the final sale amount according to Section 3.7. The percentage and discount amount shall be stored with the sale and reflected consistently in payment and gross profit calculations.
FR-09	US009	Best-selling products report. The system shall allow the owner to select a start date and an end date. It shall total the units sold for each product in completed sales within the inclusive period, including credit sales, and list products in descending order of units sold. Customer debt payments shall not add units. If the period contains no completed sales, the console shall display an informative message.
FR-10	US010	Age-verification reminder. Before completing each cash or credit sale, the system shall display a reminder and request confirmation that the cashier has performed the age check. The sale shall be completed only after confirmation and successful

		validation of the other sale requirements. Canceling shall leave stock, customer balances, and cash records unchanged.
FR-11	All user stories	JSON persistence. The system shall save successful changes to products, stock movements, suppliers, purchase orders, sales, customers, payments, and cash records in local JSON files. It shall restore saved records when the application starts. Canceled or invalid operations shall leave stored records unchanged. File loading or saving errors shall produce a clear message identifying the unsuccessful operation.
FR-12	All user stories	Input validation and correction. The system shall validate required information, numeric formats, permitted ranges, identifiers, referenced records, quantities, and dates before completing an operation. Invalid input shall produce an explanatory console message and allow correction or cancellation. Validation failures shall leave existing business records unchanged.

5. Non-Functional Requirements

The following requirements establish verifiable quality conditions for the first version of Liquor Store System. They complement the functional requirements and business rules.

5.1 Reference Test Conditions

Performance verification shall use one active application session on the computer selected for the academic demonstration.

The reference dataset shall contain 100 products and 1,000 completed sales, with up to five product entries per sale. The computer specifications, Java version, and dataset used shall be recorded with the verification results.

5.2 Quality Requirements

Requirement ID	Quality Attribute	Requirement and Verification
NFR-01	Usability	Console menus shall display numbered options, descriptive prompts, and a way to return to the main menu. Successful operations and cancellations shall produce clear confirmation messages. Verification shall include walkthroughs of product registration, sales, customer payments, and cash reconciliation.
NFR-02	Numerical accuracy	Monetary and percentage calculations shall follow the formulas and rounding rules in Section 3. Automated tests shall compare results with independently calculated expected values, including discounts, gross profit,

		margins, and cash differences. Results shall match at the specified precision.
NFR-03	Persistence reliability	After a successful save and normal application shutdown, restarting shall restore the saved field values and relationships. Existing JSON files that cannot be read or contain invalid data shall produce an error message and shall not be silently replaced with empty records. Verification shall include saving, restarting, and comparing restored records, together with an invalid-file test.
NFR-04	Error handling	Expected input errors, including nonnumeric input, invalid menu options, missing required values, and invalid ranges, shall produce explanatory messages and allow correction or cancellation without terminating the application. File-operation failures shall identify the unsuccessful operation. The application shall not report that data was saved when saving failed.
NFR-05	Maintainability and testability	Console interaction, business calculations and validation, and JSON storage shall be organized into separate classes with defined responsibilities. Business-rule methods shall be testable without manual keyboard input or changes to the application's business data files. Verification shall include code inspection and execution of the unit tests.
NFR-06	Performance	Under the reference conditions in Section 5.1, product lookups, sale-total calculations, and best-selling report generation shall each complete within two seconds after receiving the required input. Verification shall measure elapsed execution time and exclude time spent waiting for the user to type.
NFR-07	Execution and local operation	The project shall include instructions identifying the required Java version, dependencies, and steps to run it in NetBeans. After installation, the ten business functionalities and local JSON storage shall operate without an Internet connection. Verification shall include building and running the project using the documented configuration and checking its workflows without network access.

6. External Interface Requirements

This section describes the interfaces required for the first version of Liquor Store System. They shall support the functions defined in the Product Backlog through console interaction and local JSON storage.

6.1 User Interface

The application shall use a text-based console interface. Menus, prompts, confirmations, and error messages shall use consistent Spanish wording.

The user shall interact with the application by entering menu options, product codes, quantities, monetary amounts, percentages, and dates when requested.

6.1.1 Main Menu and Navigation

The main menu shall provide access to the following operations:

Option	Console Label	Related User Stories
1	Productos e inventario	US001
2	Alertas de stock	US002
3	Proveedores y pedidos	US003
4	Ventas	US004, US008, US010
5	Clientes, créditos y abonos	US005
6	Cierre de caja	US006
7	Ganancias y márgenes	US007
8	Productos más vendidos	US009
0	Salir	General navigation

Discount entry and age verification shall be included in the sales workflow.

Each submenu shall display its available options and an option to return to the main menu. An invalid menu selection shall produce an explanatory message and allow another selection.

Operations that change business records shall provide an opportunity to confirm or cancel before the changes are saved.

6.1.2 Input Formats

Prompts shall identify the information requested and show the expected format when necessary.

Input Type	Expected Format
Menu options	The number corresponding to a displayed option.
Product, customer, and supplier identifiers	The code or identifier associated with the relevant record.
Stock quantities	Whole numbers, following the permitted ranges in Section 3.
Monetary amounts	Numbers using a decimal point and no currency symbol, such as 8 or 8.50, with no more than two decimal places.
Discount percentages	Numbers entered without the percentage symbol, such as 0, 10, or 12.50, following Section 3.7.
Dates entered by the user	YYYY-MM-DD, such as 2026-10-07.

Invalid input shall produce a message identifying the problem and the expected correction. For example, a stock movement with an invalid quantity shall request a positive whole number.

The application shall allow correction or cancellation without saving an incomplete operation.

6.1.3 Console Outputs

The application shall present results through labeled values or simple lists.

Product consultations shall identify products and their available quantities. Low-stock results shall also show the minimum stock level.

Sales consultations shall show the transaction date, responsible seller, products, quantities, discount, final amount, and payment type.

Customer account consultations shall show credit purchases, payments, and outstanding balances.

Cash reconciliation shall show the selected date, expected cash, counted cash, and difference.

Gross profit results shall show the sale amount, purchase cost, gross profit, and gross profit margin.

Best-selling product reports shall show product identifiers, names, and units sold during the selected period.

Monetary outputs shall display two decimal places and identify the currency as USD. Percentages shall display two decimal places and the percentage symbol.

When a consultation has no matching records, the console shall display an informative message.

6.1.4 Sales Workflow

The sales interface shall support the following sequence:

1. Identify the responsible seller and enter the products and quantities.
2. Enter the authorized discount percentage, using 0 when no discount applies.
3. Select cash or credit as the payment type. A credit sale shall require an identified customer.
4. Display a summary containing the products, quantities, prices, subtotal, discount, final amount, seller, and payment type.
5. Display the age-verification reminder and request the cashier's confirmation.
6. Allow final confirmation or cancellation of the sale.

A sale shall be recorded only after its required information, stock availability, age confirmation, and final confirmation have been validated.

Canceling shall leave stock, customer balances, and cash records unchanged.

6.2 Hardware Interfaces

The application shall use the computer's standard keyboard for input, screen for console output, and local storage for JSON files.

The first version shall operate on the single computer selected for development and the academic demonstration.

Hardware verification shall use the reference conditions recorded for the non-functional requirements.

6.3 Software Interfaces

The application shall execute in a Java environment compatible with the project's documented configuration.

NetBeans shall be used for development and academic execution. The required Java version, dependencies, and execution instructions shall be recorded in the project documentation.

Business information shall be exchanged with the local storage component through JSON reading and writing operations. The JSON library and its version shall be identified in the project's dependency configuration.

6.4 Local Data File Interface

The application shall read and write UTF-8 JSON files in the local data folder identified in the project instructions.

File names, record structures, identifiers, and relationships shall be defined in Section 7.

Users shall create, consult, and update business records through the console options. These operations shall not require manual editing of JSON files.

Successful saves shall be acknowledged through a confirmation message. Loading or saving failures shall produce an explanatory message identifying the unsuccessful operation.

Existing files containing unreadable or invalid data shall be handled according to NFR-03 and shall not be silently replaced with empty records.

6.5 Communication Interfaces

The first version shall perform its business operations and store its records locally on one computer.

External servers, network connections, and communication with other applications shall not be required for its operation.

7. Data Requirements

The application shall store the information required by the ten user stories using records and lists in local JSON files. Purchase order items, sale items, and cash expenses shall be stored inside their corresponding parent records.

7.1 Data Types and Conventions

The following logical data types shall be used:

- Text for product codes, names, contact information, descriptions, and internal status values.
- Positive integers for automatically assigned record identifiers.
- Whole numbers for stock quantities, minimum stock levels, and transaction quantities.
- Decimal numbers for costs, prices, monetary amounts, and percentages.
- Dates stored as text in YYYY-MM-DD format.
- Boolean values for age-verification confirmation.
- Lists for records containing multiple items.

Product codes shall be entered by the user and shall be unique and nonempty. Where a record uses a numeric identifier, the application shall assign it automatically and maintain its uniqueness within that record type after restarting.

Identifiers shall remain unchanged after record creation.

JSON field names and internal codes shall use English. Their client-facing labels shall use Spanish. For example, PENDING and RECEIVED shall be displayed as Pendiente and Recibido.

Names, descriptions, and contact information entered by the user shall retain their original spelling and language.

7.2 Minimum Data Records

Record	Required Data
Product	productCode, name, purchaseCost, sellingPrice, stock, minimumStock
StockMovement	movementId, date, productCode, movementType, quantity
Supplier	supplierId, name, contact
PurchaseOrder	orderId, supplierId, orderDate, status, receivedDate, items
PurchaseOrderItem	productCode, quantity
Sale	saleId, date, seller, paymentType, customerId, items, subtotal, discountPercentage, discountAmount, totalAmount, ageVerified
SaleItem	productCode, productName, quantity, unitPurchaseCost, unitSellingPrice
Customer	customerId, name

CustomerPayment	paymentId, customerId, date, amount
DailyCashRecord	date, openingCash, countedCash, expenses
CashExpense	description, amount

The following conditions shall apply:

- Stock movement types shall be ENTRY or EXIT. Their quantities shall be positive whole numbers.
- Purchase order statuses shall be PENDING or RECEIVED. receivedDate shall remain empty until receipt is confirmed.
- Sale payment types shall be CASH or CREDIT.
- The seller shall identify the owner or employee responsible for the sale.
- A CREDIT sale shall require an existing customerId. A CASH sale may omit it.
- Only completed sales shall be stored. Their ageVerified value shall be true.
- Each DailyCashRecord shall correspond to one unique date. countedCash may remain empty until the user records the physical count.
- Each cash expense shall belong to the date of its parent DailyCashRecord.

7.3 Relationships and Historical Values

Each purchase order shall reference an existing supplier. Its items shall reference registered products.

Each sale item shall reference a registered product and preserve the product name, purchase cost per unit, and selling price per unit used when the sale was completed.

unitSellingPrice shall represent the unit price before the discount applied to the complete sale. The sale header shall preserve the discount and final amount.

Later changes to a product's name, cost, or selling price shall not modify the historical values stored with completed sales.

Credit sales and customer payments shall reference existing customers. A payment shall reduce the customer's account balance without creating another sale.

Stock movements shall reference registered products. Their records shall document changes already reflected in the stored stock quantity. Loading these records shall not apply the movements again.

7.4 Calculated Information

The application shall calculate the following information from stored records:

- Customer outstanding balance: total credit sales for the customer minus recorded customer payments.

- Gross profit and gross profit margin: calculated from the amounts and historical costs stored with each completed sale.
- Expected closing cash: calculated from opening cash and the cash operations recorded for the selected date.
- Cash difference: calculated after countedCash has been entered.
- Best-selling products: calculated by grouping and totaling units from completed sales within the selected period.
- Low-stock alerts: calculated by comparing each product's current stock with its minimumStock.

These calculations shall follow Section 3.

7.5 JSON File Organization

The application shall use a local folder named data within its documented working directory.

Each file shall contain a JSON array of records.

File Name	Content
products.json	Products and their current stock quantities and minimum levels.
stock_movements.json	Recorded stock entries and exits.
suppliers.json	Supplier information.
purchase_orders.json	Purchase orders, including their items and receipt status.
sales.json	Completed sales, including their items, historical values, discounts, and age confirmation.
customers.json	Customer identifiers and names.
payments.json	Customer debt payments.
cash_records.json	Daily opening cash, physical cash counts, and nested cash expenses.

Files shall use UTF-8 encoding. Monetary values shall be stored as numbers without currency symbols and displayed according to Section 3.1.

Users shall access these records through the application menus.

7.6 Loading, Saving, and Data Validation

On the first run, when no previous data store exists, the application shall initialize the data folder and empty record lists.

On subsequent runs, it shall load the stored records before enabling business operations. Loading shall check identifier uniqueness, required references, permitted status values, and the applicable numeric and date rules.

If a required file from an existing data store is missing, unreadable, or invalid, the application shall report the problem in Spanish and end startup without replacing the existing records with empty data.

Successful business operations shall save their affected records. Invalid or canceled operations shall leave stored information unchanged.

If saving fails, the application shall report the unsuccessful operation and shall not display a successful-save confirmation.

8. Assumptions, Dependencies and Limitations

This section establishes the conditions under which the first version of Liquor Store System is expected to operate.

8.1 Assumptions

The following assumptions apply to this version:

- The application will serve one liquor store and operate through one active session on one computer.
- The owner and employee are trusted operators. The responsible seller will be selected during sales registration.
- The owner will provide product information, purchase costs per unit, labeled selling prices, stock quantities, and minimum stock levels.
- Operators will enter information about deliveries, customer payments, expenses, and physical cash counts accurately.
- Products will be managed as whole units. Quantities purchased in bulk will be entered as the corresponding number of individual units.
- The owner will authorize discounts through business instructions followed by the cashier. The application will validate and calculate the entered percentage.
- The cashier will perform the customer's age check before confirming it in the application.
- The computer's local date will be configured correctly for recording transactions.
- The owner and employee will use a shared daily cash balance. Daily reconciliation will combine their cash sales, customer debt payments, and recorded cash expenses.

8.2 Software and Environment Dependencies

Dependency	Purpose	Configuration Status
Java Development Kit	Compile and execute the Java application.	The exact version shall be confirmed and recorded in the project configuration and execution instructions.
NetBeans	Develop, run, and demonstrate the academic project.	The version used for development and demonstration shall be recorded.
JSON library	Read and write the records defined in Section 7.	The library name and version shall be selected, checked for compatibility with the chosen Java version, and declared in the project dependencies.
JUnit	Execute automated unit	The version shall be selected and declared in

	tests for business calculations and validation rules.	the project dependencies.

The application shall have read and write access to its documented local data folder.

All three team members shall use compatible project configurations. The selected dependency versions and execution instructions shall be available before formal verification begins.

8.3 First Version Limitations

8.3.1 Interaction and Access

The first version will use a console interface with Spanish menus, prompts, messages, and results.

Seller selection will record the identity declared by the operator. Password authentication and automatic enforcement of different user permissions are outside this version's scope.

Access to the computer and its data folder will be controlled by the owner.

8.3.2 Inventory and Supplier Operations

Inventory quantities will reflect the operations recorded by users. Physical stock counts and delivery verification remain operator responsibilities.

Purchase orders will support Pending and Received statuses. Receipt confirmation will represent delivery of the complete order.

8.3.3 Sales and Payments

Sales will be recorded as cash or credit transactions. Customer debt payments will represent cash received and will be recorded separately.

Discounts will be percentages applied to the complete sale subtotal according to Section 3.7.

Cancellation will be available before a sale is completed. Editing completed sales, processing returns, and issuing refunds are outside the current scope.

8.3.4 Financial Results

Gross profit and gross profit margin will follow Section 3.8. These results describe the relationship between selling amounts and product purchase costs.

Operating expenses will be included in cash reconciliation. The application will not produce a complete net-profit statement.

The application will record internal business operations. Electronic tax invoicing and integrations with external payment services are outside this version's scope.

8.3.5 Data Storage

Records will be stored locally in the JSON files defined in Section 7.

Persistence verification will cover successful saves, normal shutdown, restarting, and the specified file-error handling.

Automatic backups and recovery from computer or storage-device failure are outside the current implementation scope.

8.4 Requirements Changes

Changes to business rules or functionality shall be reflected in the affected backlog stories, SRS requirements, acceptance criteria, and tests.

Changes to the overall project scope shall also be reflected in the Project Definition.

Document revisions shall be recorded in the version history so that the team can identify the specification used for implementation and verification.

9. Verification and Requirements Traceability

This section defines how the requirements of Liquor Store System will be verified and connected to the Product Backlog.

The following activities and test counts represent planning. Execution results shall be recorded after implementation.

9.1 Verification Methods

The team shall use the following verification methods:

- Requirements review to check consistency among the Project Definition, Product Backlog, SRS, and diagrams.
- Automated unit tests using JUnit to verify business calculations, validation rules, and changes to in-memory records.
- Integration tests to verify JSON persistence and interactions among related operations.
- Console walkthroughs to verify menus, prompts, Spanish messages, confirmations, and cancellations.
- Performance measurements under the reference conditions in Section 5.1.
- Code inspection to verify the separation of responsibilities and the source-code conventions in Section 2.1.

Unit tests shall use prepared inputs and expected results without requiring manual keyboard input or modifying the application's business data files.

9.2 Unit Test Planning and Responsibilities

The academic minimum shall be 90 automated unit tests, with at least 30 implemented by each team member.

The initial planning target shall be 100 distinct unit test cases, organized as ten cases for each user story. Cases shall cover applicable valid inputs, invalid inputs, boundaries, calculations, cancellations, and changes to business records.

Each case shall verify a distinct expected behavior. Multiple assertions within one case shall not be counted as separate tests.

The planned distribution shall be:

Team Member	Primary Stories	Primary Cases	Additional US010 Cases	Planned Total
Acurio	US001, US002, US006	30	4	34
Armas	US004, US005, US007	30	3	33
Ayala	US003, US008, US009	30	3	33
Total	All ten stories	90	10	100

Development responsibilities shall remain as defined in the Product Backlog. Test authoring for US010 shall be shared among the team members.

9.3 Functional Requirements Traceability

Unit test identifiers shall follow this format:

UT-US001-01

The identifier shall indicate the related user story and case number. The planned range for each story shall run from 01 to 10.

User Story	Functional Requirement	Business Rules	Main Verification Focus
US001	FR-01	Section 3.2	Product registration, unique codes, permitted values, stock entries, stock exits, and insufficient-stock rejection.
US002	FR-02	Section 3.3	Minimum stock validation and alerts below, at, and above the threshold, including changes after

			stock updates.
US003	FR-03	Section 3.4	Supplier and order validation, Pending status, receipt confirmation, stock increases, and prevention of repeated receipt.
US004	FR-04	Sections 3.2 and 3.5	Sale validation, quantities, totals, responsible seller, stock reduction, cancellation, and preservation of historical values.
US005	FR-05	Section 3.6	Credit balances, partial and full payments, payment history, and rejection of invalid or excessive payments.
US006	FR-06	Section 3.9	Opening cash, daily cash receipts, expenses, exclusion of unpaid credit sales, and surplus or shortage calculations.
US007	FR-07	Sections 3.1 and 3.8	Gross profit, margin, discounts, multiple units, rounding, invalid calculation inputs, and historical costs.
US008	FR-08	Sections 3.1 and 3.7	Valid percentages, percentage boundaries, discount rounding, final sale amounts, and rejection of invalid discounts.
US009	FR-09	Section 3.10	Inclusive date boundaries, quantity aggregation, descending ranking, credit sales, invalid periods, and empty results.
US010	FR-10	Section 3.11	Age confirmation for each sale, completion and cancellation outcomes, and preservation of stock and balances when confirmation is absent.

The complete case catalog shall identify the specific inputs and expected result for every planned unit test.

9.4 Shared Requirements Verification

FR-11 and FR-12 shall apply across the ten user stories.

FR-11 shall be verified through integration tests using a separate temporary data folder. These checks shall include:

- Saving valid records and loading them into a new application instance.
- Comparing restored fields, identifiers, relationships, and historical sale values.
- Confirming that canceled or invalid operations leave stored records unchanged.
- Checking the specified behavior for missing, unreadable, or invalid files in an existing data store.
- Checking that unsuccessful saves produce an error instead of a success confirmation.

FR-12 shall be verified through unit tests for validation rules and console walkthroughs for input correction and cancellation.

Integration tests and manual walkthroughs shall be recorded separately from the unit test count.

9.5 Non-Functional Requirements Verification

Requirement	Verification Method and Evidence
NFR-01	Console walkthroughs showing numbered menus, return options, descriptive prompts, and operation confirmations.
NFR-02	Automated comparisons with independently calculated expected monetary and percentage results.
NFR-03	Save-and-reload integration tests and invalid-file checks in a temporary folder, including comparisons of restored records.
NFR-04	Invalid-input walkthroughs and file-error checks showing explanatory messages and the specified recovery or termination behavior.
NFR-05	Code inspection and execution of business-rule tests without manual console input or changes to business data files.
NFR-06	Recorded execution times for the operations and dataset defined in Section 5.1, together with the computer and Java configuration.
NFR-07	Building and running the project using its documented configuration and checking local workflows without an Internet connection after installation.

9.6 Test Records and Acceptance

Each test record shall include:

- Case identifier and related requirement.
- Responsible team member.
- Preconditions and input data.
- Expected result.
- Actual result.
- Execution status and relevant evidence.

Cases shall remain marked as Not Executed until they have been run. Executed cases shall be marked as Passed or Failed according to their actual results.

Acceptance of the first version shall require:

- Verification of all ten user stories and their acceptance criteria.
- At least 90 passing automated unit tests, with at least 30 implemented by each team member.
- Successful execution of all implemented unit tests.
- Completion of the applicable integration tests, console walkthroughs, and non-functional checks.
- Correction of failures that contradict the specified requirements.
- Documentation of the Java, NetBeans, JSON library, and JUnit versions used.
- Recorded verification results and instructions for repeating the checks.

10. References and Appendices

10.1 References

The following documents support this specification:

R01. Logical Lords. *Project Definition — Liquor Store System.* Reviewed project document describing the business problem, proposed solution, and operational context.

R02. Logical Lords. *Product Backlog — Liquor Store System.* Reviewed project document containing user stories US001–US010, priorities, responsibilities, and acceptance criteria.

R03. ISO, IEC, and IEEE. (2018). *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.* General reference for requirements engineering and specification practices. Official IEEE description.

R04. Bray, T., Ed. (2017). *RFC 8259 — The JavaScript Object Notation (JSON) Data Interchange Format.* Internet Engineering Task Force. Technical reference for the JSON format used for local persistence. Official RFC publication.

The official documentation corresponding to the selected Java, NetBeans, JSON library, and JUnit versions shall be included in the project's technical references when those versions are confirmed.

10.2 Appendix A Glossary

Term	Definition
SRS	Software Requirements Specification. The document defining the application's required behavior, operating conditions, and quality requirements.
US	User Story. A business need expressed from a user's perspective.
FR	Functional Requirement. A specified behavior or capability of the application.
NFR	Non-Functional Requirement. A verifiable quality condition or operating requirement.
JDK	Java Development Kit. The development tools required to compile and run the Java project.
JSON	JavaScript Object Notation. The text format used to store the application's records in local files.
Cashier	The person registering a sale. This responsibility may be performed by the owner or employee.
Stock	The recorded quantity of available product units.
Minimum stock level	The threshold used to identify products requiring restocking.
Credit sale	A completed sale whose final amount becomes part of a customer's outstanding balance.

Customer payment	Money received to reduce an existing customer debt.
Gross profit	The final sale amount after discounts minus the purchase cost of the units sold.
Gross profit margin	Gross profit expressed as a percentage of the final sale amount.
Cash reconciliation	Comparison of expected cash with physically counted cash for a selected date.
Unit test	An automated check of a business operation or rule using prepared inputs and expected results.
Integration test	A check of interactions between components, such as saving and restoring records through JSON files.
Console walkthrough	A step-by-step check of the application's menus, prompts, outputs, and workflows.

10.3 Supporting Appendices

The following supporting documents shall accompany or be referenced by the project submission as they are completed.

Appendix	Supporting Document	Required Content	Current Status
B	Use Case Diagram	Actors and use cases corresponding to US001–US010, including the relationships between sales, discounts, and age verification.	To be updated to match this specification.
C	Class Diagram	Classes, attributes, operations, and relationships supporting the data and behavior specified in this SRS.	To be updated to match this specification.
D	Verification Case Catalog	Test identifiers, related requirements, responsibilities, preconditions, inputs, expected results, and execution results when available.	To be developed according to Section 9.
E	Configuration and Execution Guide	Confirmed tool and dependency versions, project setup, NetBeans execution steps, test execution steps, and local data-folder instructions.	To be completed when the project configuration is confirmed.

Supporting documents shall remain consistent with this SRS. Their identifiers and revision information shall be recorded when they are included in the submission.

Planned verification cases shall remain distinguishable from executed cases and their actual results.